ESCUELA SUPERIOR POLITÉCNICA AGROPECUARIA DE MANABÍ
MANUEL FÉLIX LÓPEZ

CARRERA DE DESARROLLO DE SOFTWARE
MODALIDAD EN LÍNEA • SEGUNDO SEMESTRE "A"

INFORME DE PRÁCTICA AUTÓNOMA
TRABAJO AUTÓNOMO (TA)

TEMA: "IMPLEMENTACIÓN INTEGRAL DE ESTRUCTURAS LINEALES Y CASO ASIGNADO EN FORO"

ASIGNATURA:	ESTRUCTURAS DE DATOS Y ALGORITMOS (CDS-0202
DOCENTE TITULAR:	ING. LUIGGI ALEXANDER JALCA SALTOS
ESTUDIANTE:	ALEX PATRICIO VAZQUEZ MACHAY
CASO ASIGNADO EN EL FORO:	TDA PILA (STACK - LIFO)
FECHA MÁXIMA DE ENTREGA:	01 DE NOVIEMBRE DEL 2026
ENLACE CLIKABLE DIRECTO AL RESPOSITORIO GIT	https://github.com/winnova2026EC/EDA-2N-ESPAM-APVM.git
PERÍODO ACADÉMICO:	OCTUBRE 2026 – FEBRERO 2027

Calceta, Manabí, Ecuador
2026

Contenido

TEMA: “IMPLEMENTACIÓN INTEGRAL DE ESTRUCTURAS LINEALES Y CASO ASIGNADO EN FORO” 1
1. INTRODUCCIÓN 3
2. OBJETIVO GENERAL 3
Justificación técnica: 3
3. EVIDENCIAS GRÁFICAS Y ANALÍTICAS COMENTADAS POR FASE 4
3.1. CASO ASIGNADO: TDA PILA (STACK – LIFO). 4
Descripción del problema 5
Requerimientos del sistema 5
Justificación de la estructura seleccionada 5
3.1.1 MÉTODO SCQA – TDA PILA (STACK–LIFO) 6
3.1.2. FORO 6
3.2 Diagramas de Arquitectura y Memoria 8
3.3 Desarrollo de Código JAVA 8
3.3.1 Captura comentada del módulo de - TDA PILA (STACK - LIFO) 8
3.3.2 Capturas comentadas de la implementación de TODAS las estructuras lineales 9
4. CONCLUSIONES TÉCNICAS 14

1. INTRODUCCIÓN

Este portafolio técnico tiene como finalidad demostrar los conocimientos adquiridos en la asignatura de Estructuras de Datos y Algoritmos, mediante la implementación práctica de las principales estructuras de datos lineales utilizando el lenguaje de programación Java.

Durante el desarrollo del proyecto se implementaron arreglos estáticos y dinámicos, matrices, listas enlazadas simples, dobles y circulares, pilas bajo el principio LIFO y colas circulares FIFO. Cada estructura permite comprender diferentes formas de almacenar, organizar y administrar información, considerando sus operaciones y complejidad temporal.

Además, se desarrolla el caso asignado en el Foro de Debate Técnico de la Clase 06, correspondiente al **TDA Pila (Stack – LIFO)**, aplicado a un mecanismo de Deshacer (Undo) y Rehacer (Redo) en un editor de texto. Mediante esta aplicación se demuestra cómo las pilas permiten recuperar modificaciones siguiendo el orden inverso en que fueron realizadas.

Finalmente, este portafolio reúne el análisis del problema, los diagramas de arquitectura y memoria, las implementaciones en Java y las evidencias de ejecución, fortaleciendo los conocimientos prácticos y el razonamiento algorítmico necesarios para el desarrollo de software.

2. OBJETIVO GENERAL

Implementar de manera manual las principales estructuras de datos lineales utilizando el lenguaje de programación Java, con el propósito de comprender su funcionamiento, organización en memoria y aplicación en la solución de problemas informáticos, mediante el desarrollo de arreglos, matrices, listas enlazadas, pilas LIFO y colas circulares FIFO.

Justificación técnica:

La implementación manual de las estructuras de datos lineales en Java permite comprender cómo se almacenan, organizan y manipulan los datos dentro de un programa. A través de este portafolio se desarrollaron arreglos estáticos y dinámicos, matrices, listas simplemente

enlazadas, doblemente enlazadas y circulares, pilas LIFO y colas circulares FIFO, sin utilizar colecciones predefinidas de Java para reemplazar estas estructuras.

Como aplicación práctica, se desarrolló el caso asignado en el foro de la Clase 06, correspondiente al **TDA Pila (Stack–LIFO)**, mediante un editor de texto que permite deshacer (Undo) y rehacer (Redo) modificaciones. Para su funcionamiento se utilizan dos pilas que almacenan los estados del texto y permiten recuperar las modificaciones siguiendo el principio de que el último elemento ingresado es el primero en salir.

Esta implementación permite comprender las operaciones push, pop y peek, cuya complejidad temporal es O(1), así como la importancia de seleccionar una estructura de datos adecuada para resolver un problema de programación.

3. EVIDENCIAS GRÁFICAS Y ANALÍTICAS COMENTADAS POR FASE

3.1. CASO ASIGNADO: TDA PILA (STACK – LIFO).

Aplicación en un editor de texto mediante las funciones Deshacer (Undo) y Rehacer (Redo).



Descripción del problema

En un editor de texto, los usuarios realizan diferentes modificaciones, como escribir, agregar palabras o cambiar el contenido de un documento. Durante este proceso pueden cometer errores o necesitar recuperar una modificación anterior. Por esta razón, se requiere un mecanismo que permita deshacer y rehacer las acciones realizadas, manteniendo el orden correcto de las modificaciones.

Requerimientos del sistema

1. **Registrar modificaciones:** almacenar los estados del documento cada vez que el usuario realiza una nueva escritura.
2. **Deshacer acciones (Undo):** recuperar el estado anterior del documento, comenzando por la última modificación realizada.
3. **Rehacer acciones (Redo):** permitir recuperar una modificación que anteriormente fue deshecha.
4. **Administrar dos historiales:** utilizar una pila para las acciones deshechas y otra para las acciones que pueden rehacerse.
5. **Controlar el historial:** cuando se realiza una nueva modificación después de deshacer, limpiar la pila de rehacer para evitar recuperar estados que ya no corresponden al documento actual.
6. **Mostrar el documento:** permitir visualizar el contenido actualizado después de cada operación.

Justificación de la estructura seleccionada

Se seleccionó el TDA Pila (Stack), basado en el principio LIFO (Last In, First Out), porque permite recuperar las modificaciones en orden inverso al que fueron realizadas. Esta característica es necesaria para el funcionamiento de las opciones Deshacer y Rehacer.

La implementación utiliza dos pilas y las operaciones principales push() y pop(), que permiten guardar y recuperar los estados desde la cima de la estructura en tiempo constante O(1). Por esta razón, la pila resulta adecuada para administrar el historial de modificaciones del editor de texto de manera ordenada y sencilla.

3.1.1 MÉTODO SCQA – TDA PILA (STACK–LIFO)

S – SITUACIÓN

En un editor de texto, los usuarios realizan modificaciones constantemente, como escribir palabras, agregar información o corregir el contenido de un documento. Es necesario contar con una herramienta que permita recuperar los cambios realizados.

C – COMPLICACIÓN

Durante la edición pueden cometerse errores o realizar modificaciones que posteriormente se desean revertir. Sin un mecanismo adecuado, recuperar el contenido anterior o volver a aplicar una modificación puede resultar complicado, especialmente cuando existen varias acciones consecutivas.

Q – PREGUNTA

¿Cómo implementar un mecanismo que permita deshacer y rehacer las modificaciones de un editor de texto, respetando el orden en que fueron realizadas?

A – RESPUESTA

Se propone utilizar el TDA Pila (Stack), basado en el principio LIFO (Last In, First Out), mediante dos pilas: una para almacenar los estados anteriores del documento y otra para guardar los estados deshechos. De esta manera, las operaciones push() y pop() permiten recuperar las modificaciones en orden inverso, facilitando el funcionamiento de las opciones Deshacer (Undo) y Rehacer (Redo).

3.1.2. FORO

Participación

Re: Grupo 1- Arreglos estáticos y matrices

de ALEX PATRICIO VAZQUEZ MACHAY - miércoles, 21 de octubre de 2026, 19:12

Estimados compañeros del Grupo 1:

Su propuesta de utilizar una matriz estática es adecuada para un cine, porque permite consultar y actualizar el estado de cada asiento mediante sus coordenadas de fila y columna en tiempo O(1). Sin embargo, desde el enfoque del TDA Pila (Stack LIFO), considero que nuestra estructura podría complementar su sistema al implementar una función para deshacer la última reserva realizada.

Por ejemplo, cada vez que se confirma una reserva, el sistema podría guardar la operación en una pila. Si se necesita cancelar la última reserva, se utiliza pop() para recuperar la operación más reciente y revertirla, mientras que push() permite registrar nuevas operaciones. Estas operaciones básicas tienen una complejidad O(1), siempre que se trabaje directamente con la cima de la pila.

Una pila no sustituye a la matriz para consultar cualquier asiento de manera directa, pero sí puede servir para administrar un historial de operaciones y revertir la última reserva cuando sea necesario. De esta forma, ambas estructuras podrían complementarse: la matriz administra los asientos y la pila mantiene el historial de cambios.

Enlace permanente Mostrar mensaje anterior

Replica

Grupo 4 — TDA Pila (Stack LIFO)

de ALEX PATRICIO VAZQUEZ MACHAY - miércoles, 21 de octubre de 2026, 18:38

Justificación técnica

Elegimos el TDA Pila (Stack), basado en el principio LIFO (Last In, First Out), para implementar las funciones Deshacer (Undo) y Rehacer (Redo) de un editor de texto. Esta estructura es adecuada porque permite revertir las acciones en orden cronológico inverso: la última modificación realizada es la primera que se deshace. Para ello, se utilizan dos pilas: una almacena las acciones ejecutadas y otra las acciones revertidas. Al deshacer, la acción se retira de la primera pila y se coloca en la segunda; al rehacer, se realiza el proceso contrario.

Las operaciones básicas push (apilar), pop (desapilar) y peek (consultar la cima) tienen una complejidad temporal O(1). Por ello, las pilas permiten administrar el historial de acciones sin buscar una modificación específica dentro de toda la secuencia. Cuando se realiza una nueva modificación después de deshacer, se limpia la pila Rehacer para mantener la coherencia del historial.

Pseudocódigo

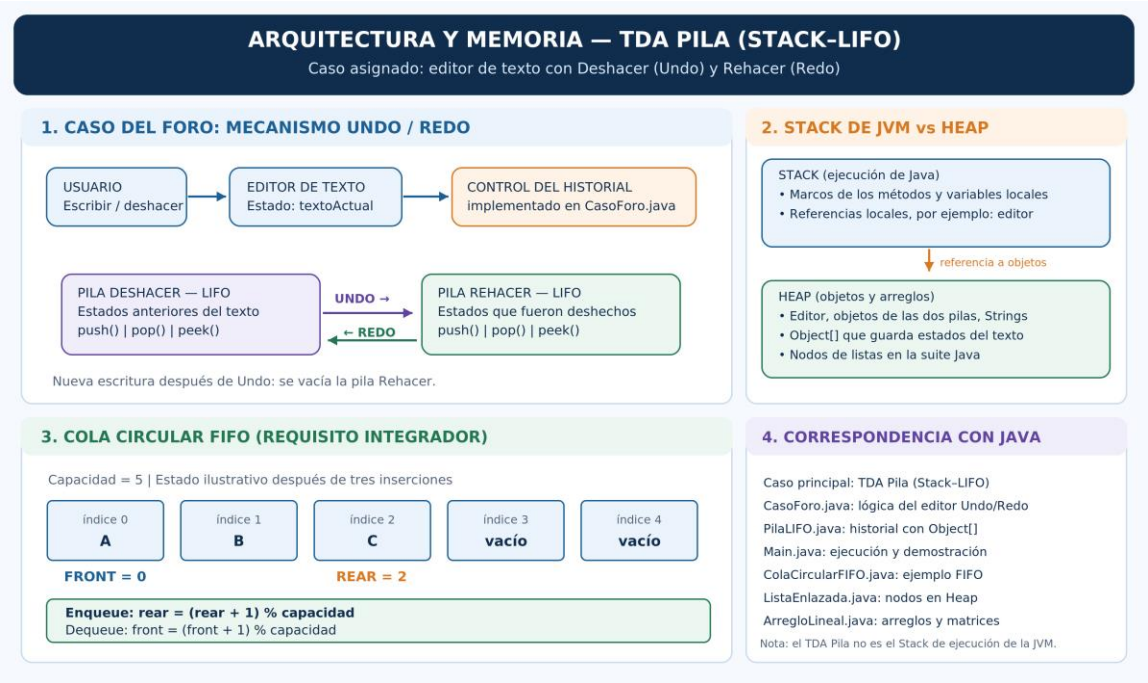
```
DESHACER:
  si pilaDeshacer no está vacía:
    accion = pop(pilaDeshacer)
    revertir(accion)
    push(pilaRehacer, accion)

REHACER:
  si pilaRehacer no está vacía:
    accion = pop(pilaRehacer)
    aplicar(accion)
    push(pilaDeshacer, accion)

NUEVA_ACCION(accion):
  aplicar(accion)
  push(pilaDeshacer, accion)
  vaciar(pilaRehacer)
```

Conclusión: la pila es una solución adecuada porque respeta el orden inverso de las acciones y permite gestionar el historial mediante operaciones básicas O(1), suponiendo que la aplicación de cada acción y su operación inversa tengan costo constante.

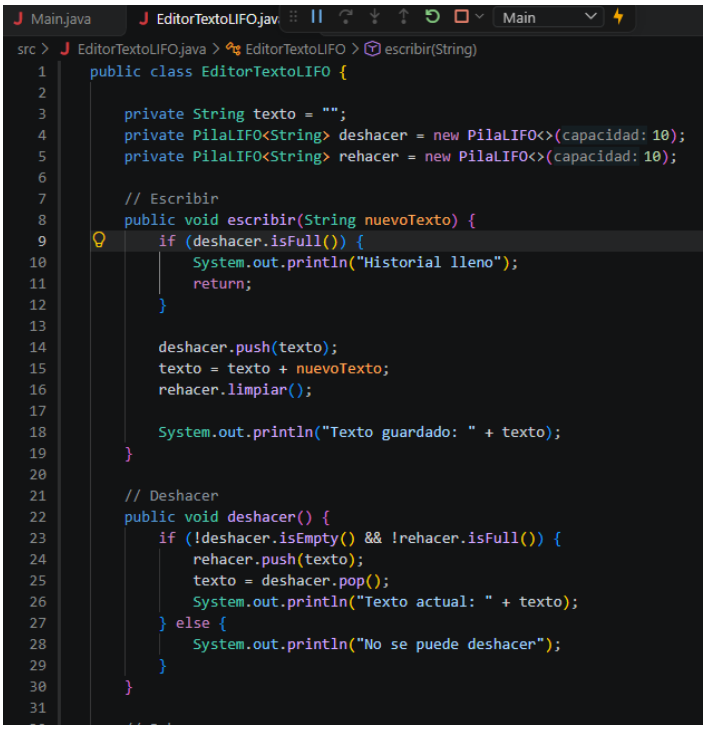
3.2 Diagramas de Arquitectura y Memoria



3.3 Desarrollo de Código JAVA

3.3.1 Captura comentada del módulo de - TDA PILA (STACK - LIFO)

EditorTextoLIFO.java (Grupo 4)



```java
public class EditorTextoLIFO {

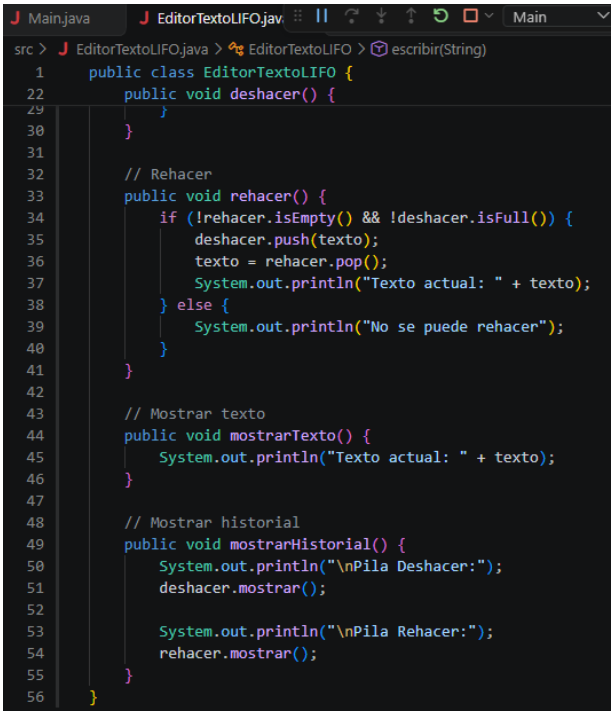
    private String texto = "";
    private PilaLIFO<String> deshacer = new PilaLIFO<>(capacidad: 10);
    private PilaLIFO<String> rehacer = new PilaLIFO<>(capacidad: 10);

    // Escribir
    public void escribir(String nuevoTexto) {
        if (deshacer.isFull()) {
            System.out.println("Historial lleno");
            return;
        }

        deshacer.push(texto);
        texto = texto + nuevoTexto;
        rehacer.limpiar();

        System.out.println("Texto guardado: " + texto);
    }

    // Deshacer
    public void deshacer() {
        if (!deshacer.isEmpty() && !rehacer.isFull()) {
            rehacer.push(texto);
            texto = deshacer.pop();
            System.out.println("Texto actual: " + texto);
        } else {
            System.out.println("No se puede deshacer");
        }
    }
```




```java
public class EditorTextoLIFO {
    public void deshacer() {
        }
    }

    // Rehacer
    public void rehacer() {
        if (!rehacer.isEmpty() && !deshacer.isFull()) {
            deshacer.push(texto);
            texto = rehacer.pop();
            System.out.println("Texto actual: " + texto);
        } else {
            System.out.println("No se puede rehacer");
        }
    }

    // Mostrar texto
    public void mostrarTexto() {
        System.out.println("Texto actual: " + texto);
    }

    // Mostrar historial
    public void mostrarHistorial() {
        System.out.println("\nPila Deshacer:");
        deshacer.mostrar();

        System.out.println("\nPila Rehacer:");
        rehacer.mostrar();
    }
}
```

3.3.2 Capturas comentadas de la implementación de TODAS las estructuras lineales

ArregloLineal.java




```java
public class ArregloLineal {

    // ARREGLO ESTATICO
    private int[] arregloEstatico;
    private int cantidadEstaticos;

    // ARREGLO DINAMICO
    private int[] arregloDinamico;
    private int cantidadDinamicos;

    // MATRIZ
    private int[][] matriz;

    public ArregloLineal() {
        arregloEstatico = new int[5];
        cantidadEstaticos = 0;

        arregloDinamico = new int[2];
        cantidadDinamicos = 0;

        matriz = new int[3][3];
    }

    // Insertar en arreglo estatico
    public boolean insertarEstatico(int valor) {

        if (cantidadEstaticos == arregloEstatico.length) {
            System.out.println("Arreglo estatico lleno.");
            return false;
        }

        arregloEstatico[cantidadEstaticos++] = valor;
        return true;
    }

    // Acceder por indice
    public int obtenerEstatico(int indice) {
```



```
src > ArregloLineal.java > ArregloLineal > matriz
2   public class ArregloLineal {
26      public boolean insertarEstatico(int valor) {
35      }
36
37      // Acceder por indice
38      public int obtenerEstatico(int indice) {
39
40          if (indice < 0 || indice >= cantidadEstaticos) {
41              throw new IndexOutOfBoundsException("Indice invalido.");
42          }
43
44          return arregloEstatico[indice];
45      }
46
47      // Insertar en arreglo dinamico
48      public void insertarDinamico(int valor) {
49
50          if (cantidadDinamicos == arregloDinamico.length) {
51              aumentarCapacidad();
52          }
53
54          arregloDinamico[cantidadDinamicos++] = valor;
55      }
56
57      // Aumentar manualmente la capacidad
58      private void aumentarCapacidad() {
59
60          int nuevaCapacidad = arregloDinamico.length * 2;
61
62          int[] nuevoArreglo = new int[nuevaCapacidad];
63
64          for (int i = 0; i < cantidadDinamicos; i++) {
65              nuevoArreglo[i] = arregloDinamico[i];
66          }
67
68          arregloDinamico = nuevoArreglo;
69
70          System.out.println("Capacidad ampliada a: " + nuevaCapacidad
```



```
2   public class ArregloLineal {
57      // Aumentar manualmente la capacidad
58      private void aumentarCapacidad() {
59
60          int nuevaCapacidad = arregloDinamico.length * 2;
61
62          int[] nuevoArreglo = new int[nuevaCapacidad];
63
64          for (int i = 0; i < cantidadDinamicos; i++) {
65              nuevoArreglo[i] = arregloDinamico[i];
66          }
67
68          arregloDinamico = nuevoArreglo;
69
70          System.out.println("Capacidad ampliada a: " + nuevaCapacidad);
71      }
72
73      // Obtener elemento del arreglo dinamico
74      public int obtenerDinamico(int indice) {
75
76          if (indice < 0 || indice >= cantidadDinamicos) {
77              throw new IndexOutOfBoundsException("Indice invalido.");
78          }
79
80          return arregloDinamico[indice];
81      }
82
83      // Insertar valor en una matriz
84      public void insertarMatriz(int fila, int columna, int valor) {
85
86          if (fila < 0 || fila >= matriz.length ||
87              columna < 0 || columna >= matriz[0].length) {
88
89              throw new IndexOutOfBoundsException("Posicion invalida.");
90          }
91
92          matriz[fila][columna] = valor;
93      }
94
```

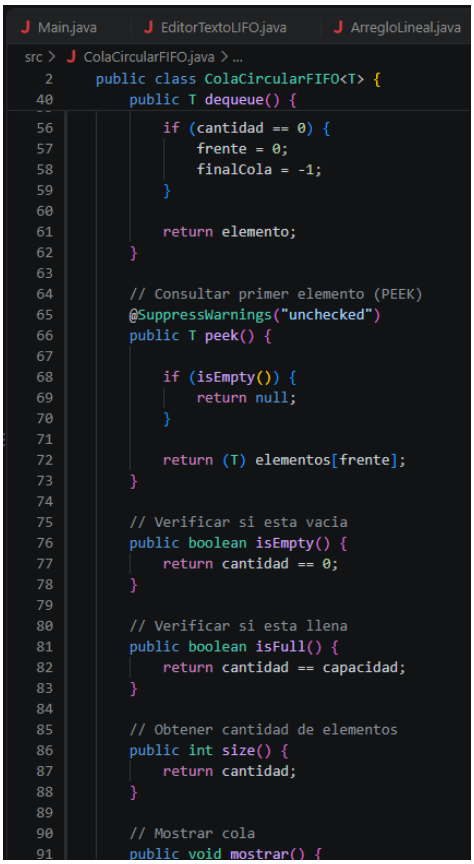
ListaEnlazada.java



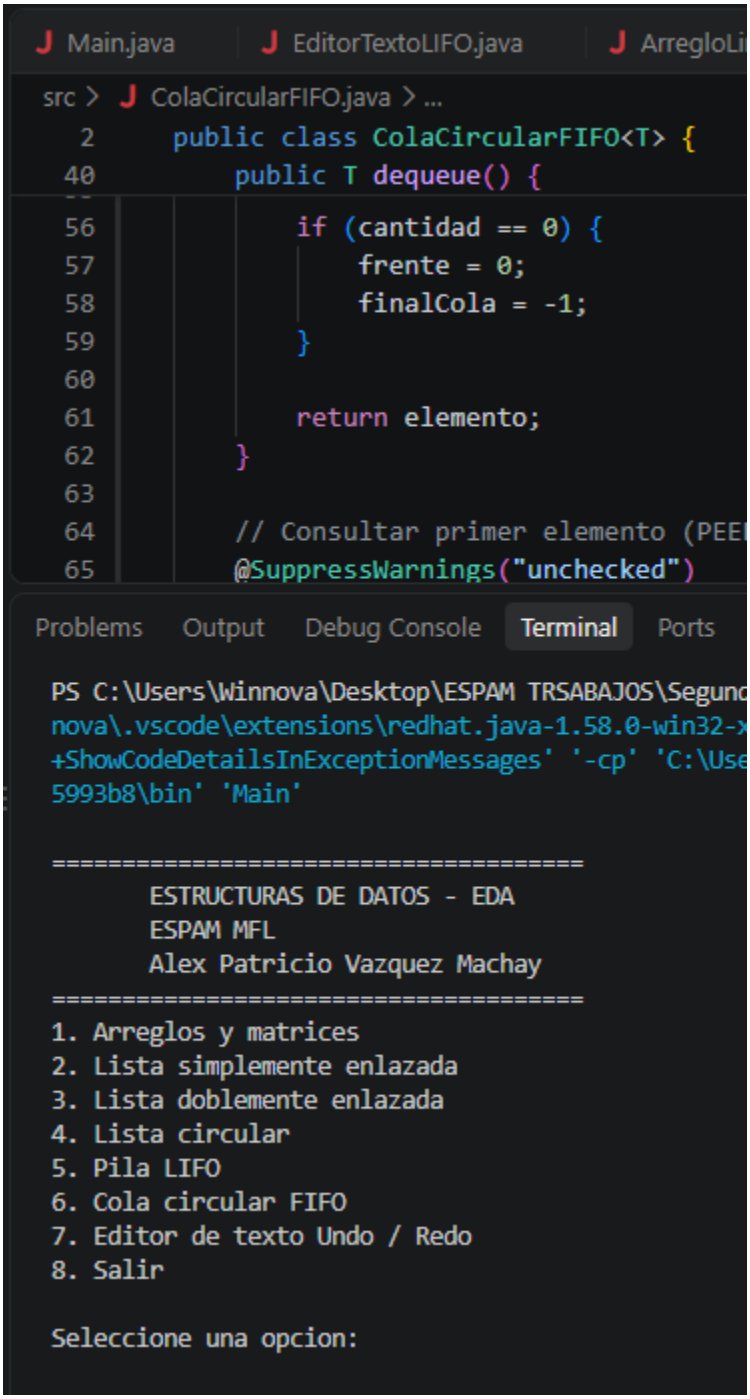


ColaCircularFIFO.java





Main.java


Main.java
EditorTextoLIFO.java
ArregloLi
src > ColaCircularFIFO.java > ...
2 public class ColaCircularFIFO<T> {
40 public T dequeue() {
56 if (cantidad == 0) {
57 frente = 0;
58 finalCola = -1;
59 }
60
61 return elemento;
62 }
63
64 // Consultar primer elemento (PEE
65 @SuppressWarnings("unchecked")
Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\Winnova\Desktop\ESPAM TRSABAJOS\Segund
nova\.vscode\extensions\redhat.java-1.58.0-win32-x
+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Use
5993b8\bin' 'Main'
ESTRUCTURAS DE DATOS - EDA
ESPAM MFL
Alex Patricio Vazquez Machay
1. Arreglos y matrices
2. Lista simplemente enlazada
3. Lista doblemente enlazada
4. Lista circular
5. Pila LIFO
6. Cola circular FIFO
7. Editor de texto Undo / Redo
8. Salir
Seleccione una opcion:

4. CONCLUSIONES TÉCNICAS

Durante el desarrollo del portafolio se implementaron diferentes estructuras de datos lineales en Java, comprendiendo sus características y funcionamiento. Se identificó que los arreglos permiten acceso directo en O(1), mientras que las listas enlazadas facilitan la inserción y eliminación mediante nodos, aunque su recorrido requiere O(n). Las pilas y colas permiten realizar sus operaciones principales en O(1).

En el caso asignado del foro, se aplicó el TDA Pila (Stack–LIFO) para desarrollar las funciones Deshacer (Undo) y Rehacer (Redo) de un editor de texto, utilizando dos pilas para guardar y recuperar modificaciones. Esta experiencia permitió fortalecer los conocimientos sobre estructuras de datos, programación Java y análisis de complejidad Big-O.